```python
#BFS
from collections import deque

def bfs (graph, start, goal):
    q = deque([(start, [start])])
    visited = set([start])
    while q:
        node, path = q.popleft()
        if node == goal:
            return path
        for nei in graph.get(node ,[]):
            if nei not in visited:
                visited.add(nei)
                q.append((nei, path + [nei]))

    return None

def main():
    graph = {
        "A" : ["B", "C", "D"],
        "D" : [],
        "B" : ["E", "F"],
        "E" : [],
        "F" : [],
        "C" : ["G", "H"],
        "G" : ["I"],
        "H" : [],
        "I" : []
    }

    result = bfs(graph, "A", "I")

    if result != None:
        print(result)
    else:
        print("No path")

if __name__ == "__main__":
    main()



#DFS


def DFS(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    while stack:
```

```
        node, path = stack.pop()
        if node == goal:
            return path
        if node in visited:
            continue
        visited.add(node)
        for nei in reversed(graph[node]):
            if nei not in visited:
                stack.append((nei, path + [nei]))
    return None

def main():
    graph = {
        "A" : ["B", "C", "D"],
        "D" : [],
        "B" : ["E", "F"],
        "E" : [],
        "F" : [],
        "C" : ["G", "H"],
        "G" : ["I"],
        "H" : [],
        "I" : []
    }

    result = DFS(graph, "A", "D")

    if result != None:
        print(result)
    else:
        print("No path")

if __name__ == "__main__":
    main()

#UCS
import heapq
def UCS(graph, start, goal):
    pq = [(0, start, [start])]
    best = {start: 0}
    while pq:
        g, node, path = heapq.heappop(pq)
        if node == goal:
            return g, path

        for nei, w in graph[node]:
            ng = g + w
            if nei not in best or ng < best[nei]:
                best[nei] = ng
                heapq.heappush(pq, (ng, nei, path +[nei]))
```

```
    return None

def main():
    graph = {
        "A": [("B", 1), ("C", 2), ("D", 3)],
        "B": [("F", 4), ("E", 5)],
        "C": [],
        "D": [],
        "E": [],
        "F": []
    }

    result = UCS(graph, "A", "F")
    if result != None:
        print(result)
    else:
        return None

if __name__ == "__main__":
    main()

#Greedy_Best_First_Search
import heapq

def greedy(graph, h, start, goal):
    pq = [(h[start], start, [start])]
    visited = set()
    while pq:
        _, node, path = heapq.heappop(pq)
        if node == goal:
            return [(n, h[n]) for n in path]
        if node in visited:
            continue
        visited.add(node)
        for nei in graph[node]:
            if nei not in visited:
                heapq.heappush(pq, (h[nei], nei, path + [nei]))
    return None

def main():
    graph = {
        "A": ["B", "C"],
        "B": [],
        "C": ["D"],
        "D": []
    }

    h = {
        "A": 2,
```

```
        "B": 2,
        "C": 1,
        "D": 0
    }

    result = greedy(graph, h, "A", "D")

    if result is None:
        print("No path")
    else:
        print(result)

if __name__ == "__main__":
    main()

#A*
import heapq

def Astar(graph, h, start, goal):
    pq = [(h[start], 0, start, [start])]
    best = {start: 0}
    while pq:
        f, g, node, path = heapq.heappop(pq)
        if node == goal:
            return [(g ,n, h[n]) for n in path]
        for nei, w in graph[node]:
            ng = g + w
            if nei not in best or ng < best[nei]:
                best[nei] = ng
                nf = ng + h[nei]
                heapq.heappush(pq, (nf, ng, nei, path + [nei]))
    return None

def main():
    graph = {
        "A": [("B", 2), ("C", 3), ("D", 4)],
        "B": [("E", 4)],
        "C": [("F", 3)],
        "D": []
    }

    h = {
        "A": 5,
        "B": 4,
        "C": 3,
        "D": 2,
        "E": 1,
        "F": 0
    }
```

```
    result = Astar(graph, h, "A", "F")

    if result:
        print(result)
    else:
        print("No solution")

if __name__ == "__main__":
    main()

#GA
import random

def genetic_algorithm(pop, fitness_fn, select_fn, crossover_fn, mutate_fn,
gens=100):
    for _ in range(gens):
        pop = sorted(pop, key=fitness_fn, reverse=True)    # best first
        new_pop = [pop[0]]                                 # elitism

        while len(new_pop) < len(pop):
            p1 = select_fn(pop, fitness_fn)
            p2 = select_fn(pop, fitness_fn)
            c1, c2 = crossover_fn(p1, p2)
            new_pop.append(mutate_fn(c1))
            if len(new_pop) < len(pop):
                new_pop.append(mutate_fn(c2))

        pop = new_pop

    return max(pop, key=fitness_fn)

#Selection
#Roulette
def roulette_wheel_selection(pop, fitness_fn):
    total_fitness = sum(fitness_fn(ind) for ind in pop)
    pick = random.uniform(0, total_fitness)
    current = 0

    for ind in pop:
        current += fitness_fn(ind)
        if current >= pick:
            return ind

    return pop[-1]

#Tournament
def tournament_selection(pop, fitness_fn, k=3):
    contestants = random.sample(pop, k)
```

```
    return max(contestants, key=fitness_fn)

#RankBased
def rank_based_selection(pop, fitness_fn):
    ranked = sorted(pop, key=fitness_fn)
    weights = list(range(1, len(ranked) + 1))
    return random.choices(ranked, weights=weights, k=1)[0]

#CrossOver
#SinglePoint
def single_point_crossover(p1, p2):
    point = random.randint(1, len(p1) - 1)
    c1 = p1[:point] + p2[point:]
    c2 = p2[:point] + p1[point:]
    return c1, c2

#TwoPoint
def two_point_crossover(p1, p2):
    a, b = sorted(random.sample(range(1, len(p1)), 2))
    c1 = p1[:a] + p2[a:b] + p1[b:]
    c2 = p2[:a] + p1[a:b] + p2[b:]
    return c1, c2

#UniformCrossover
def uniform_crossover(p1, p2):
    c1, c2 = "", ""
    for x, y in zip(p1, p2):
        if random.random() < 0.5:
            c1 += x
            c2 += y
        else:
            c1 += y
            c2 += x
    return c1, c2
#CylclicCrossOver
def cyclic_crossover(p1, p2):
    n = len(p1)
    c1 = [None] * n
    c2 = [None] * n
    visited = [False] * n

    cycle = 0
    while not all(visited):
        start = visited.index(False)
        i = start
        while not visited[i]:
            visited[i] = True
            if cycle % 2 == 0:
                c1[i] = p1[i]
```

```
                c2[i] = p2[i]
            else:
                c1[i] = p2[i]
                c2[i] = p1[i]
            i = p1.index(p2[i])
        cycle += 1

    return c1, c2

#PartiallyMaped(PMX)
def cyclic_crossover(p1, p2):
    n = len(p1)
    c1 = [None] * n
    c2 = [None] * n
    visited = [False] * n

    cycle = 0
    while not all(visited):
        start = visited.index(False)
        i = start
        while not visited[i]:
            visited[i] = True
            if cycle % 2 == 0:
                c1[i] = p1[i]
                c2[i] = p2[i]
            else:
                c1[i] = p2[i]
                c2[i] = p1[i]
            i = p1.index(p2[i])
        cycle += 1

    return c1, c2

#OrderedCrossover
def ox_crossover(p1, p2):
    n = len(p1)
    a, b = sorted(random.sample(range(n), 2))

    c1 = [None] * n
    c2 = [None] * n

    c1[a:b] = p1[a:b]
    c2[a:b] = p2[a:b]

    def fill(child, parent, a, b):
        pos = b
        for gene in parent[b:] + parent[:b]:
            if gene not in child:
                if pos == n:
```

```
                    pos = 0
                child[pos] = gene
                pos += 1
        return child

    c1 = fill(c1, p2, a, b)
    c2 = fill(c2, p1, a, b)

    return c1, c2


#Fitness
#NQueen
def n_queens_fitness(board):
    n = len(board)
    attacks = 0

    for i in range(n):
        for j in range(i + 1, n):
            if board[i] == board[j]:
                attacks += 1
            if abs(board[i] - board[j]) == abs(i - j):
                attacks += 1

    return -attacks   # higher is better
import random

pop = [
    [0, 1, 2, 3],
    [1, 3, 0, 2],
    [2, 0, 3, 1],
    [3, 2, 1, 0]
]

best = genetic_algorithm(
    pop,
    n_queens_fitness,
    tournament_selection,
    two_point_crossover,
    mutate,
    gens=50
)

print(best)
print(n_queens_fitness(best))

#8Puzzle
def puzzle_fitness(state, goal):
    score = 0
```

```
    for i in range(len(state)):
        if state[i] != 0 and state[i] != goal[i]:
            score += 1
    return -score

#MazeFitnes
def maze_fitness(position, goal):
    return -(abs(position[0] - goal[0]) + abs(position[1] - goal[1]))

#ChessFitness
def chess_fitness(board_value):
    return board_value

def chess_fitness(board):
    piece_values = {
        'P': 1, 'N': 3, 'B': 3, 'R': 5, 'Q': 9, 'K': 100
    }

    score = 0
    for row in board:
        for piece in row:
            if piece == '.':
                continue
            if piece.isupper():
                score += piece_values[piece.upper()]
            else:
                score -= piece_values[piece.upper()]
    return score
```